

# Векторные представления

- ▶ *Векторное представление* (embedding) — сопоставление произвольному объекту некоторого числового вектора в пространстве фиксированной размерности
- ▶ Наиболее известный вид – векторные представления слов (word embedding)
- ▶ Векторы могут обладать разнообразными полезными свойствами, отражать близость объектов в разных смыслах
- ▶ Для слов это может быть семантическая близость

# Зачем нужны векторные представления

В современных подходах эмбединги используются в качестве признаков для решения почти любых задач машинного обучения

**В текстовой аналитике это:**

1. выделение именованных сущностей (NER)
2. выделение частей речи (POS-tagging)
3. машинный перевод
4. кластеризация документов
5. классификация документов, анализа тональности (sentiment)
6. ранжирование документов
7. генерация текста

# One-hot encoding

Самый простой способ кодирования категориальных признаков:

| "a" | "abbreviations" |     | "zoology" | "zoom" |
|-----|-----------------|-----|-----------|--------|
| 1   | 0               |     | 0         | 0      |
| 0   | 1               |     | 0         | 1      |
| 0   | 0               |     | 0         | 0      |
| .   | .               | ... | .         | .      |
| .   | .               |     | .         | .      |
| .   | .               |     | .         | .      |
| 0   | 0               |     | 0         | 0      |
| 0   | 0               |     | 1         | 0      |
| 0   | 0               |     | 0         | 1      |

Полученные векторы огромные и ортогональные

# SVD для получения эмбедингов слов

1. По корпусу текстов  $D$  со словарём  $T$  строим *матрицу со-встречаемостей*  $X_{|T| \times |T|}$

Возможны различные варианты учёта со-встречаемости слов:

- ▶ сумма по всей коллекции числа попаданий пары слов в окно фиксированного размера
- ▶ количество документов, хоть раз содержащих пару слов
- ▶ количество документов, хоть раз содержащих пару слов в окне

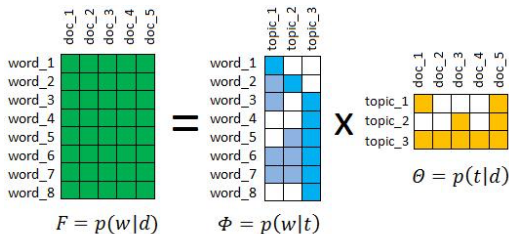
Понижаем размерность:

2. SVD-разложение:  $X = USV^T$
3. Из столбцов матрицы  $U$  выбираются первые  $K$  компонент

# Недостатки SVD

1. Относительно низкое качество получаемых представлений
2. Сложность работы с очень большой и разреженной матрицей
3. Сложность добавления новых слов/документов (решается инкрементальными методами построения)

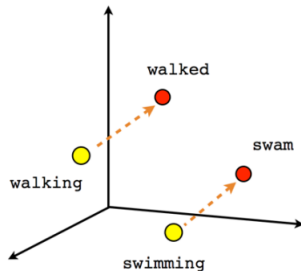
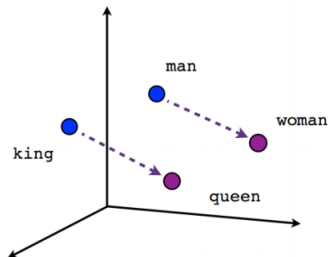
# Тематическое моделирование



- ▶ Классические тематические модели получают на вход матрицу «мешка слов» или tf-idf и строят два типа распределений:
  - ▶ слов в кластерах-темах
  - ▶ тем в документах
- ▶ По факту получается стохастическое матричное разложение
- ▶ Строки матрицы «слова-темы» можно использовать в качестве эмбедингов
- ▶ Современные реализации инкрементально обучаются на больших данных
- ▶ Как и SVD, простые ТМ не учитывают локальный контекст

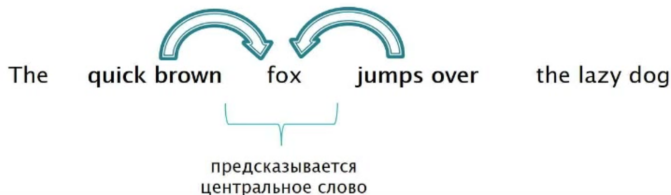
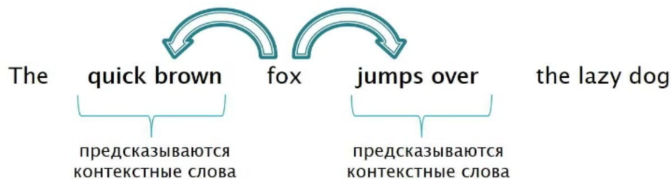
## word2vec

- ▶ **word2vec** — группа алгоритмов, предназначенных для получения вещественных векторных представлений слов
- ▶ **Идея:** «Слова со схожими значениями разделяют схожий контекст»
- ▶ Как правило, в векторном представлении семантически близкие слова оказываются рядом



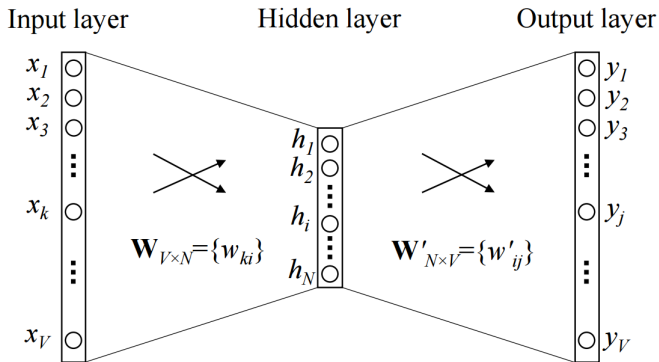
# Don't count, predict!

Две модели: **Skip-gram** и **Continuous BOW**

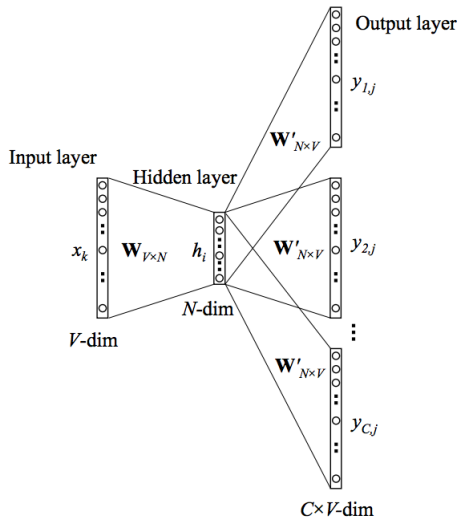


[Ссылка на источник картинки](#)

# Модель CBOW (единичный контекст)



# Модель Skip-gram



# Производительность обучения Skip-gram

Подсчёт softmax — вычислительно дорогая операция

Применяются различные методы аппроксимации:

## 1. Softmax-based

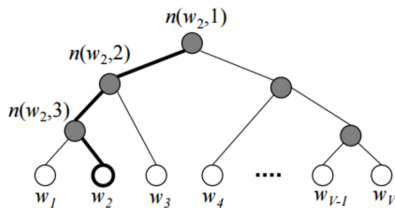
- ▶ **Иерархический softmax**
- ▶ Дифференциальный softmax
- ▶ CNN-softmax
- ▶ ...

## 2. Sampling-based

- ▶ **Negative Sampling**
- ▶ Noise Contrastive Estimation
- ▶ ...

# Иерархический softmax

- ▶ Вычисление softmax — дорогая операция,  $O(V)$
- ▶ Иерархический softmax —  $O(\log(V))$
- ▶ Строим бинарное дерево
- ▶ Связываем с каждым внутренним узлом вектор параметров
- ▶ Листья соответствуют словам



# Иерархический softmax

- ▶ Для подсчёта вероятности слова спускаемся по дереву
- ▶ На каждом шаге считаем вероятность как сигмоиду от скалярного произведения вектора весов вершины и выходного вектора контекста
- ▶ Перед применением сигмоиды скалярное произведение умножается на -1, если оценивается вероятность перехода вправо
- ▶ Полученные на всём пути до листа значения сигмoids перемножаются и дают вероятность слова
- ▶ Обучаем веса давать высокую вероятность предсказываемым словам

# Negative Sampling

- ▶ Можно изменить постановку задачи и функционал качества.
- ▶ Решаем задачу бинарной классификации:  $z = 1$  — пара  $(w, s) \in D$ ,  $z = 0$  — нет. ( $s \in c(w)$ )

$$p(z = 1 | (w, s)) = \frac{1}{1 + \exp(-v_w^T v_s)} = \sigma(v_w^T v_s)$$

- ▶ Запишем новый функционал правдоподобия:

$$\mathcal{L} = \sum_{(w,s) \in D_1} \log \sigma(v_w^T v_s) + \sum_{(w,s) \in D_2} \log \sigma(-v_w^T v_s),$$

$$D_1 = \{(w, s) : s \in c(w)\}, \quad D_2 = \{(w, c) : s \notin c(w)\}$$

## Negative Sampling

$$\mathcal{L} = \sum_{(w,s) \in D_1} \log \sigma(v_w^T v_s) + \sum_{(w,s) \in D_2} \log \sigma(-v_w^T v_s),$$

- ▶ Но множество всех отрицательных примеров отсутствует
- ▶ Выход — для каждого рассматриваемого слова  $w$  генерировать в качестве отрицательных примеров случайные слова из  $T$
- ▶ Функционал оптимизируется с помощью SGD

## word2vec и PMI

*Pointwise Mutual Information:*

$$PMI(w, c) = \log \frac{\#(w, c) |D|}{\#(w) \#(c)}$$

$$PPMI(w, c) = \max\{PMI(w, c), 0\}$$

$$SPPMI(w, c) = PPMI(w, c) - \log k$$

$k$  — число отрицательных примеров (negative samples)

Можно показать, что модель SGNS является разложением матрицы SPPMI (строки — слова, столбцы — контексты)

**Важно:** под контекстом в данном случае понимается *центральное слово*, на основании которого предсказываются стоящие рядом слова

# Global Vectors

- ▶ Строим матрицу  $X \in \mathbb{R}^{V \times V}$ ,  $x_{ij}$  — количество раз, когда слово  $i$  встречается в контексте слова  $j$  (в окне  $\leq 9$  слов)
- ▶  $P_{ij} = \frac{x_{ij}}{X_i}$  — вероятность  $j$  в контексте  $i$  ( $X_i$  — сумма  $i$ -й строки)
- ▶ Представим, что у нас есть искомые векторы слов  $w_x$
- ▶ Строим функцию  $F(w_i, w_j, \hat{w}_k)$ , показывающую, какое из слов  $i$  и  $j$  вероятнее увидеть в контексте  $k$  ( $> 1$  если  $i$  чаще)

$$F(w_i, w_j, \hat{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- ▶ Представим также, что счётчики со-встречаемостей нам неизвестны, и мы хотим их найти

- ▶ Потребуем, чтобы функция удовлетворяла следующему равенству

$$F((w_i - w_j)^T \hat{w}_k) = \frac{F(w_i^T \hat{w}_k)}{F(w_j^T \hat{w}_k)} = \frac{P_{ik}}{P_{jk}}$$

- ▶ Подходит, например,  $F(x) = \exp(x)$ , перепишем

$$w_i^T \hat{w}_k = \log(P_{ik}) = \log(x_{ik}) - \log(X_i)$$

- ▶ Вспомним, что счётчики у нас как раз есть, а векторов и сдвигов – нет:

$$w_i^T \hat{w}_k + b_i + \hat{b}_k = \log(x_{ik}), \quad b_i + \hat{b}_k = \log(X_i), \quad b_x - \text{bias}$$

- ▶ Оптимизируем с помощью AdaGrad взвешенную сумму по всем парам слов

## N-символьные эмбединги: FastText

- ▶ N-символьные эмбединги показывают более высокое качество, чем эмбединги символов
  - ▶ При этом они тоже хорошо справляются с редкими и OOV-словами
- Пример: N-граммы для слова apple (min=3, max=6):

<ap, <app, <appl, <apple, app, appl, apple,  
apple>, ppl, pple, pple>, ple, ple>, le>

**FastText** – библиотека (и алгоритм) для получения векторных представлений слов и классификации текстов

- ▶ Архитектурно такая же, как у word2vec (CBOW и skip-gram)
- ▶ Строит эмбединги символьных N-грамм
- ▶ Эмбединг слова получается усреднением эмбедингов N-грамм

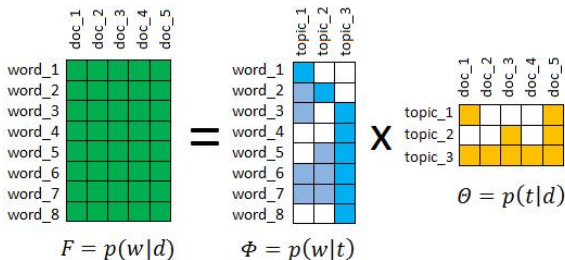
# FastText

- ▶ Для борьбы с ростом размерности признакового пространства используется *hashing trick*
- ▶ В FastText есть встроенный классификатор на основе двуслойной полносвязной сети
- ▶ Вход – эмбединг документа как сумма эмбедингов слов
- ▶ На выходе стоит иерархический софтмакс с деревом Хаффмана
- ▶ За счёт использования символьных N-грамм может из коробки хорошо работать с сырыми текстами
- ▶ Ссылки на статьи: <https://arxiv.org/pdf/1607.01759.pdf>,  
<https://arxiv.org/pdf/1607.04606.pdf>

# Эмбединги предложений и документов

- ▶ Часто в задачах текстовой аналитики нужны эмбединги не слов в документах, а самих документов или их частей
- ▶ Самый простой способ получения – взвешенная сумма эмбедингов слов (например, с tf-idf в качестве весов)
- ▶ Такой подход показывает хорошие результаты и часто используется на практике, но есть и более интересные методы
- ▶ Если есть хороший способ найти эмбединг предложения, вектор документа можно опять-таки получить усреднением по предложениям
- ▶ Качество оценивается по метрике задачи (внешний критерий) или, например, качеству поиска размеченных аналогий (внутренний)

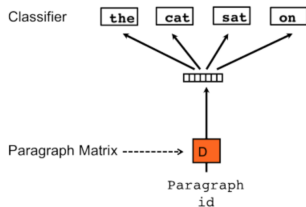
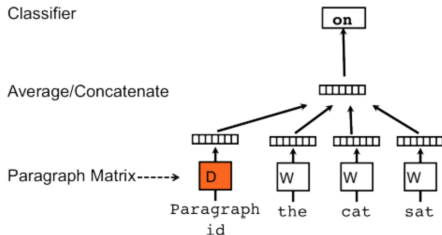
# Снова тематическое моделирование



- ▶ Классические тематические модели получают на вход матрицу «мешка слов» или tf-idf и строят два типа распределений:
  - ▶ слов в кластерах-темах
  - ▶ тем в документах
- ▶ Столбцы матрицы «темы-документы» можно использовать в качестве эмбедингов документов
- ▶ Они отражают близость документов со схожей тематикой

## doc2vec (paragraph2vec)

- ▶ **doc2vec** – дополненная архитектура word2vec для получения эмбедингов текстовых фрагментов
- ▶ Добавляем ещё одну матрицу весов для документов
- ▶ Обученные векторы для слов используются при обучении векторов новых документов



## Что дальше?

- ▶ context2vec
- ▶ USE
- ▶ ULMFIT
- ▶ ELMo
- ▶ BERT
- ▶ ERNIE
- ▶ GPT-2
- ▶ ...

**А на сегодня всё!**

# Эмбе́ддинги в неевклидовых пространствах

- Евклидово пространство имеет нулевую кривизну и плохо масштабируется для иерархических данных
- Объём пространства растёт полиномиально, что недостаточно для представления древовидных структур с экспоненциальным ростом
- Иерархические отношения (гиперонимы-гипонимы) трудно кодировать через расстояния
- **Идея:** использовать пространства с ненулевой кривизной.  
**Например:** постоянная отрицательная кривизна (гиперболические) - объём растёт экспоненциально, что естественным образом соответствует иерархиям

# Гиперболические эмбе́ддинги

## Модели пространства:

- Модель Пуанкаре: диск с метрикой  $ds^2 = \frac{4|dx|^2}{(1-|x|^2)^2}$ , геодезические — дуги окружностей
- Модель Лоренца (гиперболоид):  $x \in \mathbb{R}^{d+1}$  с  $\langle x, x \rangle_{\mathcal{L}} = -1, x_0 > 0$
- Иерархическая классификация текстов
- Представление таксономий
- Обработка данных с естественной иерархией

# Эмбе́ддинги на гиперсфере

- Размещение эмбе́ддингов на  $d$ -мерной гиперсфере
- Цель: сохранить семантическую информацию, раздвигая несвязанные векторы без использования негативных примеров
- Риманова оптимизация для дисперсии эмбе́ддингов на гиперсфере

# Гильбертовы эмбединги

## Геометрия Гильберта:

- Обобщение модели Клейна гиперболического пространства
- Метрика на симплексе:  $\rho_H^\Omega(p, q)$  — симметризация расстояния Фанка
- Преимущество: замкнутая форма расстояния, дифференцируемость

# Пуассоновы и вероятностные эмбе́ддинги

- Пуассоновская регрессия для эмбе́ддингов:
  - Применение для DNA-хранилищ: встраивание расстояния Левенштейна с пуассоновской регрессией
  - Пуассоновская матричная факторизация: эмбе́ддинги для разреженных данных с распределением Пуассона
- Вместо детерминированных векторов — вероятностные распределения
- Расстояние заменяется на правдоподобие или расхождение
- Позволяет работать с неопределённостью и разреженными данными

# Кривизна и объем

| Пространство                      | Кривизна   | Объём    | Применение             |
|-----------------------------------|------------|----------|------------------------|
| Евклидово                         | 0          | $r^d$    | Базовая задача         |
| Гиперболическое (Пуанкаре/Лоренц) | -1         | $e^{cr}$ | Иерархия, таксономии   |
| Гиперсфера                        | +1         | $r^d$    | Контрастивное обучение |
| Гильбертов симплекс               | Переменная | —        | Контрастивное обучение |

# HELM (Hyperbolic Large Language Models)

- ▶ Первые полностью гиперболические LLM с миллиардами параметров
- ▶ Архитектура: Mixture-of-Curvature Experts (HELM-MICE) — каждый "эксперт" работает в пространстве со своей кривизной для кодирования более тонкой геометрической структуры текста
- ▶ Разработаны гиперболические эквиваленты RoPE (ротационных позиционных кодировок) и RMS-нормализации
- ▶ Результат: улучшение до 4% на бенчмарках MMLU и ARC по сравнению с евклидовыми архитектурами LLaMA и DeepSeek

# Hyperbolic Vision-Language Model

- ▶ Впервые предложено полностью использовать иерархическую природу гиперболических эмбеддингов в Vision-Language моделях
- ▶ Идея: изображение — это композиция объектов, каждый со своим текстовым описанием (извлекаются из предложений с помощью моделей grounding)
- ▶ Иерархическая организация: изображения → объекты на изображении → текстовые описания через контрастное обучение и entailment-обучение
- ▶ Превосходит евклидов CLIP и предыдущие гиперболические альтернативы в zero-shot и retrieval задачах

# Literature on modern embeddings

- ▶ Tokarchuk E., Bakker H.C., Niculae V. On the Matter of Embeddings Dispersion on Hyperspheres. ICML 2024 Workshops: GRaM.
- ▶ Sundararaman N., Pragya P., Geetika S., Kiran D.D. Exploring Adaptive Curvature Techniques on Hyperbolic Embeddings for Hierarchical Text Classification. iSAI-NLP 2024.
- ▶ Chikoti S., Mehta S., Modi A. IITK at SemEval-2024 Task 4: Hierarchical Embeddings for Detection of Persuasion Techniques in Memes. SemEval 2024.
- ▶ Yessenbayev Z., Kozhirbayev Z. Use of Riemannian Distance Metric to Verify Topological Similarity of Acoustic and Text Domains. ICANN 2024.
- ▶ Li X., Matsuda T., Komaki F. Empirical Bayes Poisson matrix completion. Computational Statistics and Data Analysis, 2024.
- ▶ He N. et al. HELM: Hyperbolic Large Language Models via Mixture-of-Curvature Experts. NeurIPS 2025.
- ▶ Pal A. et al. Compositional Entailment Learning for Hyperbolic Vision-Language Models. ICLR 2025.